TruthTable: Benchmark Query Listings

Supplementary material — full synthetic and TPC-H query set

A Synthetic Benchmark Queries

To compare TruthTable on workloads where PoSQL [Spa25] and qedb [BBCRT25] baselines are applicable, we benchmark a set of synthetic queries that exercise the core relational operators in isolation: Filter, Aggregate, Join, Join (PK/FK), and Limit/Offset.

Filter

```
SELECT *
FROM {table}
WHERE a = 1 AND b = 2;
```

Aggregate

```
SELECT count(*)
FROM {table};
```

Join

```
SELECT {table1}.a, {table2}.a
FROM {table1}
JOIN {table2} ON {table1}.a = {table2}.a;
```

Join (PK/FK)

```
SELECT {table1}.a, {table2}.a
FROM {table1}
JOIN {table2} ON {table1}.a = {table2}.a;
```

Limit/Offset

```
SELECT *
FROM {table}
LIMIT 10;
```

B TPC-H Benchmark Queries

The TPC-H benchmark [PF00] is a standard decision-support workload used widely in database design and systems research. It distills business-oriented analytics into a collection of parameterized SQL queries that stress scans, joins, aggregations, and nested subqueries. To ground our evaluation in a familiar setting, we follow prior work by selecting a subset of representative queries and executing them atop the canonical schema and data-generation procedures.

For completeness, we list the exact SQL text of the queries we benchmark. For the queries used in the Poneglyph[GFN25] comparison (Q1, Q3, Q5, Q8, Q9, Q18), we additionally list the abridged variant supported by Poneglyph's implementation.

The text in each box shows the query as actually executed; we measure it against the canonical TPC-H text (specifically, the version returned by DuckDB's `tpch_queries()`) and note any project-side modifications underneath. A few modifications recur and are explained here once: (i) `extract(year from ...)` is replaced by a reference to a precomputed `*_year` column, because DataFusion does not yet plan `EXTRACT(YEAR FROM ...)`; (ii) decimal casts of the form `CAST(0.2 AS DECIMAL(15,2))` are added because our bench tables expose decimal-typed columns, so float literals require an explicit cast; (iii) some correlated subqueries are flattened to inner joins so that the benchmark exercises the same join gadget path used elsewhere, rather than planner-generated semi-joins with null right-side provenance. The abridged Poneglyph variants additionally rename every base table to `<name>_poneglyph` and drop trailing semicolons; only the further Poneglyph-specific abridgements are listed beneath each abridged query. Queries marked "*Modifications:* None" are unchanged from the canonical text.

Q1: Pricing Summary Report

```
SELECT
    l_returnflag,
    l_linestatus,
    sum(l_quantity) AS sum_qty,
    sum(l_extendedprice) AS sum_base_price,
    sum(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
    sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
    avg(l_quantity) AS avg_qty,
    avg(l_extendedprice) AS avg_price,
    avg(l_discount) AS avg_disc,
    count(*) AS count_order
FROM lineitem
WHERE l_shipdate <= CAST('1998-09-02' AS date)
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

Modifications: None.

Q1 (Poneglyph abridged): Pricing Summary Report

```
SELECT
    l_returnflag,
    sum(l_quantity) AS sum_qty,
    sum(l_extendedprice) AS sum_base_price,
```

```
    sum(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
    sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
    count(*) AS count_order
FROM lineitem_poneglyph
WHERE l_shipdate <= CAST('1998-09-02' AS date)
GROUP BY l_returnflag
ORDER BY l_returnflag
```

Modifications: Drops `l_linestatus` from `SELECT`/`GROUP BY`/`ORDER BY`; drops all three `avg(...)` aggregates (`avg_qty`, `avg_price`, `avg_disc`).

Q2: Minimum Cost Supplier

```
SELECT s_acctbal, s_name, n_name, p_partkey, p_mfgr, s_address, s_phone,
↪  s_comment
FROM part, supplier, partsupp, nation, region
WHERE p_partkey = ps_partkey
  AND s_suppkey = ps_suppkey
  AND p_size = 15
  AND s_nationkey = n_nationkey
  AND n_regionkey = r_regionkey
  AND r_name = 'EUROPE'
  AND ps_supplycost = (
      SELECT min(ps_supplycost)
      FROM partsupp, supplier, nation, region
      WHERE p_partkey = ps_partkey
        AND s_suppkey = ps_suppkey
        AND s_nationkey = n_nationkey
        AND n_regionkey = r_regionkey
        AND r_name = 'EUROPE')
ORDER BY s_acctbal DESC, n_name, s_name, p_partkey
LIMIT 100;
```

Modifications: Removes the `p_type LIKE '%BRASS'` filter.

Q3: Shipping Priority

```
SELECT l_orderkey,
       sum(l_extendedprice * (1 - l_discount)) AS revenue,
       o_orderdate,
       o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
  AND c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND o_orderdate < CAST('1995-03-15' AS date)
  AND l_shipdate > CAST('1995-03-15' AS date)
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

Modifications: None.

Q3 (Poneglyph abridged): Shipping Priority

```
SELECT l_orderkey,
       sum(l_extendedprice * (1 - l_discount)) AS revenue,
       o_orderdate,
       o_shippriority
FROM customer_poneglyph, orders_poneglyph, lineitem_poneglyph
WHERE c_mktsegment = 'BUILDING'
  AND c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND o_orderdate < CAST('1995-03-15' AS date)
  AND l_shipdate > CAST('1995-03-15' AS date)
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
```

Modifications: Removes `LIMIT 10`.

Q4: Order Priority Checking

```
SELECT o_orderpriority, count(*) AS order_count
FROM orders
INNER JOIN (
    SELECT l_orderkey
    FROM lineitem
    WHERE l_receiptdate > l_commitdate
    GROUP BY l_orderkey
) AS late_orders ON o_orderkey = l_orderkey
WHERE o_orderdate >= CAST('1993-07-01' AS date)
  AND o_orderdate < CAST('1993-10-01' AS date)
GROUP BY o_orderpriority
ORDER BY o_orderpriority;
```

Modifications: Replaces the correlated `EXISTS (SELECT * FROM lineitem WHERE l_orderkey = o_orderkey AND l_commitdate < l_receiptdate)` with an `INNER JOIN` against a derived table; functionally a semi-join $\rightarrow$ inner-join rewrite.

Q5: Local Supplier Volume

```
SELECT n_name, sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem, supplier, nation, region
WHERE c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND l_suppkey = s_suppkey
  AND c_nationkey = s_nationkey
  AND s_nationkey = n_nationkey
  AND n_regionkey = r_regionkey
  AND r_name = 'ASIA'
  AND o_orderdate >= CAST('1994-01-01' AS date)
```

```
  AND o_orderdate < CAST('1995-01-01' AS date)
GROUP BY n_name
ORDER BY revenue DESC;
```

Modifications: None.

Q5 (Poneglyph abridged): Local Supplier Volume

```
SELECT n_name, sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer_poneglyph, orders_poneglyph, lineitem_poneglyph,
     supplier_poneglyph, nation_poneglyph, region_poneglyph
WHERE c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND l_suppkey = s_suppkey
  AND c_nationkey = s_nationkey
  AND s_nationkey = n_nationkey
  AND n_regionkey = r_regionkey
  AND r_name = 'ASIA'
  AND o_orderdate >= CAST('1994-01-01' AS date)
  AND o_orderdate < CAST('1995-01-01' AS date)
GROUP BY n_name
ORDER BY revenue DESC;
```

Modifications: Table rename only.

Q6: Forecasting Revenue Change

```
SELECT sum(l_extendedprice * l_discount) AS revenue
FROM lineitem
WHERE l_shipdate >= CAST('1994-01-01' AS date)
  AND l_shipdate < CAST('1995-01-01' AS date)
  AND l_discount BETWEEN 0.05 AND 0.07
  AND l_quantity < 24;
```

Modifications: None.

Q7: Volume Shipping

```
SELECT supp_nation, cust_nation, l_year, sum(volume) AS revenue
FROM (
    SELECT n1.n_name AS supp_nation,
           n2.n_name AS cust_nation,
           l_shipdate_year AS l_year,
           l_extendedprice * (1 - l_discount) AS volume
    FROM supplier, lineitem, orders, customer, nation n1, nation n2
    WHERE s_suppkey = l_suppkey
      AND o_orderkey = l_orderkey
      AND c_custkey = o_custkey
      AND s_nationkey = n1.n_nationkey
```

```
      AND c_nationkey = n2.n_nationkey
      AND ((n1.n_name = 'FRANCE' AND n2.n_name = 'GERMANY')
        OR (n1.n_name = 'GERMANY' AND n2.n_name = 'FRANCE'))
      AND l_shipdate BETWEEN CAST('1995-01-01' AS date)
                         AND CAST('1996-12-31' AS date)
) AS shipping
GROUP BY supp_nation, cust_nation, l_year
ORDER BY supp_nation, cust_nation, l_year;
```

Modifications: `extract(year FROM l_shipdate)` → reference to precomputed column `l_shipdate_year`.

Q8: National Market Share

```
SELECT o_orderdate_year,
       sum(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END) AS
       ↪  mkt_share_num,
       sum(volume) AS mkt_share_denom
FROM (
    SELECT o_orderdate_year,
           l_extendedprice * (1 - l_discount) AS volume,
           n2.n_name AS nation
    FROM part, supplier, lineitem, orders, customer, nation n1, nation n2,
    ↪  region
    WHERE p_partkey = l_partkey
      AND s_suppkey = l_suppkey
      AND l_orderkey = o_orderkey
      AND o_custkey = c_custkey
      AND c_nationkey = n1.n_nationkey
      AND n1.n_regionkey = r_regionkey
      AND r_name = 'AMERICA'
      AND s_nationkey = n2.n_nationkey
      AND o_orderdate BETWEEN CAST('1995-01-01' AS date)
                          AND CAST('1996-12-31' AS date)
      AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_orderdate_year
ORDER BY o_orderdate_year;
```

Modifications: `extract(year FROM o_orderdate) AS o_year` → reference to precomputed column `o_orderdate_year` (output column renamed accordingly); the original `sum(CASE...) / sum(volume) AS mkt_share` ratio is split into two output columns (`mkt_share_num`, `mkt_share_denom`), so the engine no longer has to compute the division.

Q8 (Poneglyph abridged): National Market Share

```
SELECT o_orderdate_year,
       sum(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END) AS
       ↪  mkt_share_num,
       sum(volume) AS mkt_share_denom
FROM (
    SELECT o_orderdate_year,
```

```
           l_extendedprice * (1 - l_discount) AS volume,
           n2.n_name AS nation
    FROM part_poneglyph, supplier_poneglyph, lineitem_poneglyph,
         orders_poneglyph, customer_poneglyph,
         nation_poneglyph n1, nation_poneglyph n2, region_poneglyph
    WHERE p_partkey = l_partkey
      AND s_suppkey = l_suppkey
      AND l_orderkey = o_orderkey
      AND o_custkey = c_custkey
      AND c_nationkey = n1.n_nationkey
      AND n1.n_regionkey = r_regionkey
      AND r_name = 'AMERICA'
      AND s_nationkey = n2.n_nationkey
      AND o_orderdate BETWEEN CAST('1995-01-01' AS date)
                          AND CAST('1996-12-31' AS date)
      AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_orderdate_year
ORDER BY o_orderdate_year;
```

Modifications: Same rewrites as Q8 above (precomputed `o_orderdate_year`; ratio split into `mkt_share_num` / `mkt_share_denom`).

Q9: Product Type Profit Measure

```
SELECT nation, o_orderdate_year, sum(amount) AS sum_profit
FROM (
    SELECT n_name AS nation,
           o_orderdate_year,
           l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity AS
           ↪  amount
    FROM part, supplier, lineitem, partsupp, orders, nation
    WHERE s_suppkey = l_suppkey
      AND ps_suppkey = l_suppkey
      AND ps_partkey = l_partkey
      AND p_partkey = l_partkey
      AND o_orderkey = l_orderkey
      AND s_nationkey = n_nationkey
) AS profit
GROUP BY nation, o_orderdate_year
ORDER BY nation, o_orderdate_year DESC;
```

Modifications: Removes the `p_name LIKE '%green%'` filter; `extract(year FROM o_orderdate) AS o_year` → reference to precomputed column `o_orderdate_year` (output column renamed accordingly).

Q9 (Poneglyph abridged): Product Type Profit Measure

```
SELECT nation, o_orderdate_year, sum(amount) AS sum_profit
FROM (
    SELECT n_name AS nation,
           o_orderdate_year,
```

```
          l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity AS
          ↪ amount
    FROM part_poneglyph, supplier_poneglyph, lineitem_poneglyph,
         partsupp_poneglyph, orders_poneglyph, nation_poneglyph
    WHERE s_suppkey = l_suppkey
      AND ps_suppkey = l_suppkey
      AND ps_partkey = l_partkey
      AND p_partkey = l_partkey
      AND o_orderkey = l_orderkey
      AND s_nationkey = n_nationkey
) AS profit
GROUP BY nation, o_orderdate_year
ORDER BY nation, o_orderdate_year DESC;
```

Modifications: Same rewrites as Q9 above (drop `p_name LIKE '%green%'`; precomputed `o_orderdate_year`).

Q10: Returned Item Reporting

```
SELECT c_custkey, c_name,
       sum(l_extendedprice * (1 - l_discount)) AS revenue,
       c_acctbal, n_name, c_address, c_phone, c_comment
FROM customer, orders, lineitem, nation
WHERE c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND o_orderdate >= CAST('1993-10-01' AS date)
  AND o_orderdate < CAST('1994-01-01' AS date)
  AND l_returnflag = 'R'
  AND c_nationkey = n_nationkey
GROUP BY c_custkey, c_name, c_acctbal, c_phone, n_name, c_address, c_comment
ORDER BY revenue DESC
LIMIT 20;
```

Modifications: None.

Q12: Shipping Modes and Order Priority

```
SELECT l_shipmode,
       sum(CASE WHEN o_orderpriority = '1-URGENT' OR o_orderpriority =
       ↪ '2-HIGH'
                THEN 1 ELSE 0 END) AS high_line_count,
       sum(CASE WHEN o_orderpriority <> '1-URGENT' AND o_orderpriority <>
       ↪ '2-HIGH'
                THEN 1 ELSE 0 END) AS low_line_count
FROM orders, lineitem
WHERE o_orderkey = l_orderkey
  AND l_shipmode IN ('MAIL', 'SHIP')
  AND l_commitdate < l_receiptdate
  AND l_shipdate < l_commitdate
  AND l_receiptdate >= CAST('1994-01-01' AS date)
  AND l_receiptdate < CAST('1995-01-01' AS date)
```

```
GROUP BY l_shipmode
ORDER BY l_shipmode;
```

Modifications: None.

Q14: Promotion Effect

```
SELECT CAST(100.00 AS DECIMAL(15, 2)) * sum(l_extendedprice * (1 -
↪  l_discount))
       / sum(l_extendedprice * (1 - l_discount)) AS promo_revenue
FROM lineitem, part
WHERE l_partkey = p_partkey
  AND l_shipdate >= date '1995-09-01'
  AND l_shipdate < CAST('1995-10-01' AS date);
```

Modifications: Drops the `CASE WHEN p_type LIKE 'PROMO%' THEN ... ELSE 0 END` numerator entirely (so the `LIKE` filter is removed and the ratio becomes degenerate, with `sum(l_extendedprice * (1 - l_discount))` in both numerator and denominator); `100.00 *` → `CAST(100.00 AS DECIMAL(15, 2)) *`.

Q15: Top Supplier

```
WITH revenue AS (
    SELECT l_suppkey AS supplier_no,
           sum(l_extendedprice * (1 - l_discount)) AS total_revenue
    FROM lineitem
    WHERE l_shipdate >= CAST('1996-01-01' AS date)
      AND l_shipdate < CAST('1996-04-01' AS date)
    GROUP BY supplier_no
),
max_revenue AS (
    SELECT max(total_revenue) AS highest_val FROM revenue
)
SELECT s_suppkey, s_name, s_address, s_phone, total_revenue
FROM supplier
INNER JOIN revenue     ON s_suppkey = supplier_no
INNER JOIN max_revenue ON total_revenue = highest_val
ORDER BY s_suppkey;
```

Modifications: Adds a second CTE `max_revenue`; the canonical version's cross-join with a `WHERE`-equality and inline scalar subquery (`SELECT max(total_revenue) FROM revenue`) is replaced with two explicit `INNER JOIN`s (`ON s_suppkey = supplier_no` and `ON total_revenue = highest_val`).

Q17: Small-Quantity-Order Revenue

```
SELECT sum(l_extendedprice) / CAST(7.0 AS DECIMAL(15, 2)) AS avg_yearly
FROM lineitem, part
WHERE p_partkey = l_partkey
  AND p_brand = 'Brand#23'
```

```
  AND p_container = 'MED BOX'
  AND l_quantity < (
      SELECT CAST(0.2 AS DECIMAL(15, 2)) * avg(l_quantity)
      FROM lineitem
      WHERE l_partkey = p_partkey);
```

Modifications: Decimal casts on the literals `7.0` and `0.2` (`CAST(... AS DECIMAL(15, 2))`); otherwise structurally identical (the correlated subquery is preserved).

Q18: Large Volume Customer

```
SELECT c_name, c_custkey, o_orderkey, o_orderdate, o_totalprice,
↪  sum(l_quantity)
FROM customer, orders, lineitem
WHERE o_orderkey IN (
        SELECT l_orderkey
        FROM lineitem
        GROUP BY l_orderkey
        HAVING sum(l_quantity) > 300)
  AND c_custkey = o_custkey
  AND o_orderkey = l_orderkey
GROUP BY c_name, c_custkey, o_orderkey, o_orderdate, o_totalprice
ORDER BY o_totalprice DESC, o_orderdate
LIMIT 100;
```

Modifications: None.

Q18 (Poneglyph abridged): Large Volume Customer

```
SELECT c_name, c_custkey, o_orderkey, o_orderdate, o_totalprice,
       SUM(l_quantity) AS sum_l_quantity
FROM customer_poneglyph
JOIN orders_poneglyph   ON c_custkey = o_custkey
JOIN lineitem_poneglyph ON o_orderkey = l_orderkey
GROUP BY c_name, c_custkey, o_orderkey, o_orderdate, o_totalprice
ORDER BY o_totalprice DESC, o_orderdate ASC;
```

Modifications: Drops the `o_orderkey IN (SELECT l_orderkey ... HAVING sum(l_quantity) > 300)` pre-filter entirely (this is the substantive abridgement: there is no big-order pre-filter, and the `GROUP BY` now sees all orders); cross joins rewritten to explicit `JOIN ... ON` syntax; drops `LIMIT 100`; adds `sum_l_quantity` alias on `SUM(l_quantity)`.

Q19: Discounted Revenue

```
SELECT sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM lineitem, part
WHERE (p_partkey = l_partkey
       AND p_brand = 'Brand#12'
       AND p_container IN ('SM CASE', 'SM BOX', 'SM PACK', 'SM PKG')
```

```
        AND l_quantity >= 1  AND l_quantity <= 1 + 10
        AND p_size BETWEEN 1 AND 5
        AND l_shipmode IN ('AIR', 'AIR REG')
        AND l_shipinstruct = 'DELIVER IN PERSON')
    OR (p_partkey = l_partkey
        AND p_brand = 'Brand#23'
        AND p_container IN ('MED BAG', 'MED BOX', 'MED PKG', 'MED PACK')
        AND l_quantity >= 10 AND l_quantity <= 10 + 10
        AND p_size BETWEEN 1 AND 10
        AND l_shipmode IN ('AIR', 'AIR REG')
        AND l_shipinstruct = 'DELIVER IN PERSON')
    OR (p_partkey = l_partkey
        AND p_brand = 'Brand#34'
        AND p_container IN ('LG CASE', 'LG BOX', 'LG PACK', 'LG PKG')
        AND l_quantity >= 20 AND l_quantity <= 20 + 10
        AND p_size BETWEEN 1 AND 15
        AND l_shipmode IN ('AIR', 'AIR REG')
        AND l_shipinstruct = 'DELIVER IN PERSON');
```

Modifications: None.

Q20: Potential Part Promotion

```
SELECT s_name, s_address
FROM supplier
INNER JOIN nation ON s_nationkey = n_nationkey
INNER JOIN (
    SELECT ps_suppkey
    FROM partsupp
    INNER JOIN part ON ps_partkey = p_partkey
    INNER JOIN (
        SELECT l_partkey, l_suppkey,
               CAST(0.5 AS DECIMAL(15, 2)) * sum(l_quantity) AS half_sum_qty
        FROM lineitem
        WHERE l_shipdate >= CAST('1994-01-01' AS date)
          AND l_shipdate < CAST('1995-01-01' AS date)
        GROUP BY l_partkey, l_suppkey
    ) AS lineitem_half_sum
        ON ps_partkey = l_partkey
       AND ps_suppkey = l_suppkey
    WHERE ps_availqty > half_sum_qty
    GROUP BY ps_suppkey
) AS candidate_suppliers ON s_suppkey = ps_suppkey
WHERE n_name = 'CANADA'
ORDER BY s_name;
```

Modifications: Flattens the canonical triple-nested correlated subquery (`s_suppkey IN (SELECT ps_suppkey FROM partsupp WHERE ps_partkey IN (...) AND ps_availqty > (SELECT 0.5 * sum(l_quantity) FROM lineitem WHERE ...))`) into all `INNER JOIN`s, with the `lineitem` aggregate pulled out into a derived table joined on `(l_partkey, l_suppkey)`; removes the `p_name LIKE 'forest%'` filter (`part` is joined unfiltered); `0.5 *` → `CAST(0.5 AS DECIMAL(15, 2)) *`.

Execution notes. The workload covered by the framed snippets spans scan-heavy aggregates (Q1, Q6),

nested-subquery selections (Q2, Q17), join-and-sort pipelines (Q3, Q10), order-priority and shipping-mode analytics (Q4, Q12), geography-aware joins (Q5, Q7), nested aggregation queries (Q8), profit-attribution workloads (Q9), single-pass arithmetic aggregates (Q14), CTE-based top-supplier ranking (Q15), large-volume customer reporting (Q18), highly selective disjunctive predicates (Q19), and supplier-screening pipelines (Q20). Collectively, they exercise every proving gadget in TruthTable, providing a realistic baseline for system-wide tuning.

References

[BBCRT25] V. Botta, S. Bottoni, M. Campanelli, E. Ragnoli, and A. Trombetta. "qedb: Expressive and Modular Verifiable Databases (without SNARKs)". Cryptology ePrint Archive, Paper 2025/1408. 2025.

[GFN25] B. Gu, J. Fang, and F. Nawab. "PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification". In: *Proc. ACM Manag. Data* 3.1 (2025), 63:1–63:27.

[PF00] M. Poess and C. Floyd. "New TPC Benchmarks for Decision Support and Web Commerce". In: *SIGMOD Records*. SIGMOD '00 29.4 (Dec. 2000). tex.acmid= 369291 tex.issue_date= Dec. 2000 tex.numpages= 8, pp. 64–71. ISSN: 0163-5808. DOI: 10.1145/369275.369291.

[Spa25] Space and Time. *Space and Time — Proof of SQL (sxt-proof-of-sql)*. Version v0.100.0. GitHub repository (tag v0.100.0, commit 1c27b7f). Accessed 2026-02-06. May 7, 2025.